

IFN584 Assignment 1: OO Programming

- **Assessment type:** Individual project
- **Weight:** 20% of total unit grade
- **Due:** Friday 28 August (Week 6)
- **Platform:** .NET 10 (C# console application)
- **Deliverables:** C# source code archive with `README.md` and `asciinema` recording

1 Overview

In this assignment, you will design and implement a text-based C# console application for **GomokuPlus**, a tactical variant of the classic strategy board game Gomoku (Five-in-a-Row).

This project reinforces core Object-Oriented programming concepts and implementation in C#, preparing you for future assignments. Upon successful completion of this project, you will be able to:

- **Apply core OOP techniques:** Model game domain entities using well-encapsulated classes and clean abstractions.
- **Implement game state logic:** Manage grid representation, rule enforcement, move validation, and victory evaluations.
- **Handle structured I/O & exceptions:** Build an interactive terminal interface, parse user and test script inputs, and handle errors robustly.
- **Perform file operations:** Implement persistent game state (Save/Load).

The submission must be your own original work. The use of generative artificial intelligence (GenAI) tools is prohibited during this assessment. This assignment is eligible for the 48-hour late submission period and assignment extensions.

2 Task Description & System Requirements

You are required to build **GomokuPlus** as a .NET 10 C# console application running in the terminal. The required application features are as follows:

2.1 Game Modes

- Human vs. human (HvH): Two human players alternate turns on the same terminal interface.
- Human vs. computer (HvC): A human player plays against an automated AI. You are free to implement any algorithm for the AI player (including simple random moves), but it must strictly select a valid, legal move on its turn.

2.2 Turn Validation

- The application must validate all input coordinates and move choices, preventing illegal moves (e.g., placing a stone off-grid, placing on an occupied cell, or targeting a non-existent opponent stone with an Eraser).

2.3 Game State Persistence (Save / Load)

- Players can save the current game state to a file at any time and restore it later. Resuming a saved file restores board positions, turn count, current player, game mode, and remaining special stone counts.

2.4 In-Game Help System

- A built-in help menu for human players explaining move input syntax, available special stone counts, and sample commands.

3 Gameplay Mechanics

3.1 The Grid Board

- **Fixed dimensions:** The game is played on a fixed 10×10 grid (100 total cells).
- **Grid rendering:** Each cell is formatted to be three whitespace characters wide, separated by vertical pipe characters (|) and horizontal lines (+---+).
- **Coordinates:** Rows are numbered 1 to 10 (top to bottom), and columns are numbered 1 to 10 (left to right).

	1	2	3	4	5	6	7	8	9	10
	+---+---+---+---+---+---+---+---+---+---+									
1										
	+---+---+---+---+---+---+---+---+---+---+									
2										
	+---+---+---+---+---+---+---+---+---+---+									
3										
	+---+---+---+---+---+---+---+---+---+---+									
4										
	+---+---+---+---+---+---+---+---+---+---+									
5										
	+---+---+---+---+---+---+---+---+---+---+									
6										
	+---+---+---+---+---+---+---+---+---+---+									
7										
	+---+---+---+---+---+---+---+---+---+---+									
8										
	+---+---+---+---+---+---+---+---+---+---+									
9										
	+---+---+---+---+---+---+---+---+---+---+									
10										
	+---+---+---+---+---+---+---+---+---+---+									

3.2 Win Condition

A player wins instantly upon aligning 5 consecutive stones of their player symbol horizontally, vertically, or diagonally.

3.3 Stone Types & Inventory Allotment

Stone Type	Allotment per Player	Board Symbol (P1 / P2)	Mechanics & Rules
Ordinary Stone	Unlimited stones	X / O	Placed on an empty cell. Contributes toward 5-in-a-row winning alignments.
Heavy Stone	2 stones	@ / #	Placed on an empty cell. Contributes toward winning alignments. Cannot be targeted or removed by an opponent's Eraser Stone.
Eraser / Lift Stone	2 stones	N/A (Never rendered)	Targeted at an existing opponent Ordinary Stone on the grid. Instantly removes the target stone from the board, leaving the cell empty again. The Eraser Stone itself is consumed and is never displayed on the board.

4 Input Commands & Testing Mode Specification

4.1 Command Syntax

During interactive gameplay, players enter move commands using standard string formats:

- O[Row]:[Col] — Place an Ordinary Stone (e.g., O3:4).
- H[Row]:[Col] — Place a Heavy Stone (e.g., H5:5).
- E[Row]:[Col] — Execute an Eraser Stone targeting an opponent stone (e.g., E3:3).

4.2 Automated Testing Mode

Your program must support an automated script parsing mode to run automated game sequences without interactive menu prompts. The testing mode accepts a single command-line execution string:

O3:2, O3:3, O3:1, O4:3, O3:4, O5:3, E3:3, H3:3

Step-by-step breakdown:

1. P1: Ordinary stone at (3,2) → Cell (3,2) = x
2. P2: Ordinary stone at (3,3) → Cell (3,3) = o
3. P1: Ordinary stone at (3,1) → Cell (3,1) = x
4. P2: Ordinary stone at (4,3) → Cell (4,3) = o
5. P1: Ordinary stone at (3,4) → Cell (3,4) = x
6. P2: Ordinary stone at (5,3) → Cell (5,3) = o
7. P1: Eraser stone targeting (3,3) → Cell (3,3) is cleared
8. P2: Heavy stone at (3,3) → Cell (3,3) = #

For example, if we run the string from the command-line:

```
> dotnet run "03:2,03:3,03:1,04:3,03:4,05:3,E3:3,H3:3"
```

The resulting board state would output:

```

      1  2  3  4  5  6  7  8  9 10
+---+---+---+---+---+---+---+---+---+
1 |   |   |   |   |   |   |   |   |   |
+---+---+---+---+---+---+---+---+
2 |   |   |   |   |   |   |   |   |   |
+---+---+---+---+---+---+---+---+
3 | X | X | # | X |   |   |   |   |   |
+---+---+---+---+---+---+---+---+
4 |   |   | 0 |   |   |   |   |   |   |
+---+---+---+---+---+---+---+---+
5 |   |   | 0 |   |   |   |   |   |   |
+---+---+---+---+---+---+---+---+
6 |   |   |   |   |   |   |   |   |   |
+---+---+---+---+---+---+---+---+
7 |   |   |   |   |   |   |   |   |   |
+---+---+---+---+---+---+---+---+
8 |   |   |   |   |   |   |   |   |   |
+---+---+---+---+---+---+---+---+
9 |   |   |   |   |   |   |   |   |   |
+---+---+---+---+---+---+---+---+
10|   |   |   |   |   |   |   |   |   |
+---+---+---+---+---+---+---+---+

```

5 Demonstration Recording with asciinema

To verify your program functionality, you must submit a terminal recording generated using `asciinema` (asciinema.org). This lightweight terminal recorder captures terminal output cleanly without needing graphical video recording software.

5.1 Installing asciinema

For macOS terminal setups, install via Homebrew (`brew.sh`):

```
> brew install asciinema
```

On Linux (or WSL on Windows 10/11) environments:

```
> sudo apt update
> sudo apt install asciinema
```

If you are on Windows and prefer not to use WSL, you may alternatively use the `asciinema`-compatible terminal recorder `PowerSession` ([PowerSession website on GitHub](https://github.com/PowerSession/PowerSession)) that supports native Windows builds and installation:

```
> winget install PowerSession
```

You can then use the `powersession` command instead of `asciinema` in the commands below.

5.2 Recording Instructions & Mandatory Checklist

1. **Start recording:** Run the following command in your terminal before launching your game:

```
> asciinema rec StudentID.cast
```

2. **Identification step:** At the very start of the recording session, display your identification and environment details using the following commands (replacing with your student ID and full name):

```
> echo "=== STUDENT IDENTIFICATION ==="
> echo "Student ID: N12345678 | Full Name: First Last"
> whoami && hostname
> dotnet --info
```

The identification step is mandatory; missing it will result in a zero for this assignment.

3. **App execution & walkthrough (demonstrate ALL required features):**

- Build and launch your application (`dotnet run`).
- **Human vs. human (HvH) mode:** Show turn transitions, grid rendering, and input validation (e.g., entering an out-of-bounds or invalid command).
- **Win detection:** Show horizontal, vertical, and diagonal win conditions.
- **Human vs. computer (HvC) mode:** Demonstrate playing against the AI opponent.
- **Save / load persistence:** Demonstrate saving a game session mid-play, quitting, loading the saved game state, and continuing play seamlessly.
- **Special stones execution:** Demonstrate placing a Heavy Stone and executing an Eraser Stone targeting an opponent's ordinary stone (verify single-frame clearing to). Attempting to erase a Heavy Stone must show a validation error.
- **Automated testing mode:** Run the program in script-parsing mode using the test command string: `03:2,03:3,03:1,04:3,03:4,05:3,E3:3,H3:3`.

4. **Ending the recording:** Exit your application and type `exit` or press `Ctrl+D` in the terminal to stop recording.
5. **Playback verification:** Replay your recording to ensure all features are clearly visible prior to submission:

```
> asciinema play StudentID.cast
```

Note: You don't have to record everything in one go. If you need to pause and resume recording, append to the same file using:

```
> asciinema rec -a StudentID.cast
```

6 Marking Criteria (20 Points Total)

Grading is based on a 20-point scale, split across 6 categories. Each category is evaluated based on the completeness and correctness demonstrated in the recording as well as testing.

6.1 Category Breakdown

Category	Max Points	Core Evaluation Focus
1. Basic Gameplay (HvH, Win Check)	4 pts	Human vs. Human turn execution, grid rendering, input validation, and win detection (horizontal, vertical, diagonal 5-in-a-row).
2. Basic Gameplay HvC	2 pts	Computer AI opponent turn execution, valid move generation, and immediate win detection.
3. Save / Load State Persistence	2 pts	Saving state mid-game to file, quitting, restoring game state (board, turns, scores, inventories) seamlessly.
4. Special Stones Gameplay (Heavy & Eraser)	4 pts	Implementation and rule enforcement for Heavy Stones and Eraser Stones, inventory limits (2 each), single-frame execution, and error handling.
5. Automated Testing Mode	4 pts	Command-line script execution mode, turn parsing, state execution, and final grid rendering.
6. OO Architecture & Code Quality	4 pts	Object-oriented design: clear class separation, encapsulation, proper access modifiers, and clean C# syntax.

CRITICAL MARKING REQUIREMENT:

Only the requirements correctly claimed in your README.md file and clearly demonstrated in your asciinema recording file will receive marks. Even if a feature is correctly implemented in your C# source code, if it is not claimed and demonstrated/verified working in your .cast recording, no mark will be awarded for that specific feature/criterion.

7 Submission Instructions

7.1 Submission Archive Structure

Submit a single .zip archive named StudentID.zip via Canvas.

DO NOT INCLUDE BUILD OR EXECUTABLE FILES:

Your submission archive must NOT include compiled build outputs, binaries, or temporary build folders (such as bin/, obj/, .vs/, or .vscode/). Run dotnet clean before compressing your folder. Only include raw C# source code files (.cs), project files (.csproj), .cast recording, and README.md.

The archive root must contain:

1. **C# solution source directory:** Containing your `.csproj` file and all source files (`.cs`). Uncompilable or inexecutable source code cannot be marked and will receive zero marks. To ensure you have included all necessary files, unzip the archive file and verify that they build cleanly using standard .NET CLI commands:

```
> dotnet clean
> dotnet run
```

2. **README.md:** Markdown text file with student identification details and the Mandatory Feature Implementation Claim Table (see template below).
3. **Terminal recording file:** Your asciinema (or powersession) recording file named `StudentID.cast`.

7.2 Mandatory README.md Template

Your submitted `README.md` file must contain the following completed table. You must explicitly declare the working status of each requirement and provide the exact timestamp (minutes:seconds) where each feature is demonstrated in your `StudentID.cast` recording file:

```
# Assignment 1 Submission README

* **Student Name:** [Your Full Name]
* **Student ID:** [Your Student ID]
* **Unit:** IFN584 Object-Oriented Design and Development
* **Recording File:** `StudentID.cast` (replace with your own student ID)

## Feature Implementation Claim Table

| Requirement | Status | asciinema Timestamp (mm:ss) | Notes |
| :--- | :--- | :--- | :--- |
| **Human vs. human (HvH) mode** | Working / Not Working | 01:00 | N/A |
| **Human vs. computer (HvC) mode** | Working / Not Working | 02:00 | N/A |
| **Save / load state persistence** | Working / Not Working | 03:00 | N/A |
| **Heavy stone** | Working / Not Working | 04:00 | N/A |
| **Eraser stone** | Working / Not Working | 05:00 | N/A |
| **Automated testing script mode** | Working / Not Working | 06:00 | N/A |
```

Markdown (`.md`) files can be previewed natively in most code editors.

To find out the timestamp of a feature in your recording file, you can use the `asciinema` Web Player by uploading your `.cast` file. Run the following command:

```
> asciinema upload StudentID.cast
```

Then, follow the provided private link to view your recording and find the timestamps in a web browser.